

Nigerian Financial Intelligence & Analytics Engine

Backend Architecture, Processing Pipelines, API Design & ML-Ready
Infrastructure

*A senior-level engineering masterclass — from raw financial data to production-grade analytics
infrastructure*

Backend Architecture Series · Financial Intelligence Edition · Production-Grade Design

Table of Contents

PART 1 — The Problem Space

- 1.1 Why This Problem Exists
- 1.2 Why a Backend-First Architecture
- 1.3 What This System Actually Is

PART 2 — System Thinking & Core Abstractions

- 2.1 The Three Processing Machines
- 2.2 Core Domain Entities
- 2.3 Service Boundaries
- 2.4 Information Flow

PART 3 — Architecture

- 3.1 High-Level Architecture
- 3.2 Modular Directory Structure
- 3.3 Layered Architecture
- 3.4 Monolith vs Microservices Decision
- 3.5 API Architecture

PART 4 — Rule Engine & Processing Pipeline

- 4.1 What a Rule Engine Is
- 4.2 Rule Abstraction Levels
- 4.3 Scoring System Design
- 4.4 The Full Profile Generation Pipeline
- 4.5 Nigerian Context as a Domain Object

PART 5 — Database Design

- 5.1 Relational Modelling Principles
- 5.2 Entity-Relationship Design
- 5.3 SQLAlchemy Schema
- 5.4 Profile Versioning

PART 6 — Object-Oriented Design & SOLID Principles

- 6.1 Polymorphism in the System
- 6.2 SOLID Principles Applied
- 6.3 Dependency Injection in FastAPI

PART 7 — Analytics & Scoring APIs

- 7.1 API Architecture Overview
- 7.2 Financial Input APIs
- 7.3 Analytics & Scoring APIs
- 7.4 Dashboard APIs
- 7.5 Insight APIs

PART 8 — Binary Features & Vectorization

- 8.1 Why Binary Representation Matters
- 8.2 The Profile Feature Vector
- 8.3 Weighted Scoring
- 8.4 Generic Adaptability

PART 9 — Future-Proofing & ML Readiness

- 9.1 LLM Integration Architecture

9.2 pgvector & Vector Databases

9.3 Event-Driven Systems

9.4 Observability Strategy

9.5 Testing Strategy

PART 10 — Implementation Roadmap

10.1 Phase Plan

10.2 Engineering Priorities

APPENDIX — Quick Reference

Complete Rule Set

Advice Template Example

Architecture Explanation Scripts

The Problem Space

1.1 Why This Problem Exists

The Nigerian personal finance crisis is not a literacy problem. It is a structural mismatch problem. Most Nigerians who earn money understand that money should be managed. What fails them is the absence of contextualised financial intelligence — advice and analytics that account for the exact economic pressure they live inside.

Consider the structural realities:

- **Inflation erosion.** Nigeria's inflation consistently runs 20–30%+ annually. Saving in naira without modelling this is a silent wealth destroyer.
- **Dual-pressure economy.** A significant portion of working Nigerians support extended family — school fees, hospital bills, emergency contributions — a phenomenon with no Western finance equivalent.
- **Informal income structure.** A large percentage of income earners receive irregular, cash-based income. No payslip, no tax record, no credit history. Formal financial tools fail them by design.
- **Rent-first economy.** In Lagos, Abuja, and Port Harcourt, rent is typically paid annually upfront. A single month's salary must be understood in the context of a 1-year rent obligation the user is silently carrying.
- **Data and transport tax.** Internet data and daily transportation are not optional luxuries. They are operational costs of surviving in a Nigerian city and must be baked into every financial model as baseline expenses.

Most financial tools available to Nigerians are either generic Western advice (an emergency fund of 6 months is meaningless if you earn ₦60,000), Instagram investment noise, or bank apps that track spending but provide zero analytical intelligence. The missing layer is a reasoning engine that says: *given that you earn ₦80,000, pay rent, have two dependants, and live in Lagos — here is the precise reality of your financial situation and the only realistic moves available to you.*

1.2 Why a Backend-First Architecture

This system is not a chatbot, budgeting tracker, or investment platform. It is backend-driven financial analytics infrastructure. The frontend is only a visualisation layer. The backend is the actual product.

The backend does all of this:

- Collects and validates structured financial data
- Processes financial inputs through deterministic pipelines
- Derives financial metrics and computes financial scores

- Generates analytical states and produces financial insights
- Exposes dashboard-ready analytics APIs

Critical rule

The LLM never computes financial truth. The backend computes truth. The LLM only explains truth. This separation is the architectural foundation of the entire system.

1.3 What This System Actually Is

At its core, this is a profile-driven analytics engine with a natural language explanation layer available as a future integration point. It has three conceptual machines running in sequence:

```
[User Input] → [Profile Engine] → [Financial Profile]
                                     ↓
[User Question] → [Intent Router] → [Analytics Engine] → [Response]
```

The Profile Engine transforms raw human facts into structured financial attributes. The Analytics Engine consumes structured attributes and produces structured insights. The Intent Router maps natural language queries to analytics categories.

System Thinking & Core Abstractions

2.1 The Three Processing Machines

Before writing a single line of code, you identify the core abstractions — the fundamental concepts the system reasons about. Each machine has a clearly defined contract with the others.

2.2 Core Domain Entities

User

A person with identity, demographics, and a financial context.

FinancialProfile

A structured, computed representation of a user's financial reality. This is not user input — it is derived. It is the system's interpretation of the user's situation based on the rules applied to their raw data.

FinancialMetrics

Computed numerical measurements derived from the profile: debt-to-income ratio, savings rate, financial pressure score, investment readiness score, emergency fund gap.

AnalyticsState

The aggregate of all computed scores, labels, and vectors for a user at a point in time. This is what the dashboard APIs serve.

FinancialInsight

A structured response generated by the analytics engine, optionally enriched by an LLM explanation layer.

ConversationSession

A stateful interaction window linking questions and insights to a specific profile snapshot.

2.3 Service Boundaries

The system is organised around four domain services:

- **ProfileEngine** — Takes raw `UserInputData` and returns a `FinancialProfile`.
- **AnalyticsEngine** — Takes a `FinancialProfile` and returns computed `FinancialMetrics` and scores.
- **InsightEngine** — Takes `FinancialMetrics` and returns `FinancialInsight` objects.

- **IntentClassifier** — Takes a raw natural language string and returns a classified intent and extracted entities.

2.4 Information Flow

Step 1: User completes onboarding form

→ Raw UserInput collected (name, age, income, location, dependants, rent...)

Step 2: Backend receives UserInput

→ ProfileEngine evaluates each input against rule set

→ Computes derived attributes (income_level, risk_level, pressure_score...)

→ Saves FinancialProfile to DB

→ Computes binary feature vector, stores in profile_vector (JSONB)

Step 3: AnalyticsEngine runs

→ Reads FinancialProfile

→ Computes all financial metrics

→ Persists AnalyticsState

Step 4: Dashboard APIs serve AnalyticsState

→ Frontend renders charts and summaries

Step 5: User types a question

→ IntentClassifier maps question to intent category

→ InsightEngine generates insight from profile + metrics

→ (Optional) LLM explanation layer wraps insight in natural language

→ Response returned and stored under session

[illegible]

```
nigerian_finance_api/
■
■■■ main.py                # FastAPI application entrypoint
■■■ config.py              # Environment config, settings
■
■■■ api/                   # HTTP Layer – routes only, no business logic
■   ■■■ v1/
■       ■■■ auth.py        # POST /auth/register, /auth/login
■       ■■■ onboarding.py  # POST /onboarding
■       ■■■ profile.py     # GET /profile/{user_id}
■       ■■■ analytics.py   # GET /analytics/{user_id}
■       ■■■ scoring.py     # GET /scoring/{user_id}
■       ■■■ dashboard.py   # GET /dashboard/{user_id}
■       ■■■ insights.py    # POST /insights/query
```

```

■      ■■■ vectors.py          # GET /vectors/{user_id}
■
■■■ domain/                    # Core business logic – pure Python, no HTTP
■   ■■■ profile/
■   ■   ■■■ engine.py          # ProfileEngine class
■   ■   ■■■ rules.py           # Rule definitions
■   ■   ■■■ scoring.py         # Scoring and weighting logic
■   ■   ■■■ models.py          # FinancialProfile dataclass
■   ■
■   ■■■ analytics/
■   ■   ■■■ engine.py          # AnalyticsEngine class
■   ■   ■■■ metrics.py         # Financial metric computations
■   ■   ■■■ pipeline.py        # Analytics computation pipeline
■   ■   ■■■ models.py          # AnalyticsState, FinancialMetrics
■   ■
■   ■■■ insights/
■   ■   ■■■ engine.py          # InsightEngine class
■   ■   ■■■ templates.py       # Insight template registry
■   ■   ■■■ intent.py          # IntentClassifier
■   ■   ■■■ models.py          # FinancialInsight dataclass
■   ■
■   ■■■ features/
■   ■   ■■■ extractor.py        # FeatureExtractor class
■   ■   ■■■ vectorizer.py       # Profile vectorization
■   ■   ■■■ registry.py        # Feature registry
■   ■
■   ■■■ knowledge/
■       ■■■ nigerian_context.py # Nigerian financial constants
■
■■■ db/
■   ■■■ models.py              # SQLAlchemy ORM models
■   ■■■ session.py             # DB connection management
■   ■■■ repositories/
■       ■■■ user_repo.py
■       ■■■ profile_repo.py
■       ■■■ analytics_repo.py
■       ■■■ session_repo.py
■
■■■ schemas/                    # Pydantic request/response schemas
■   ■■■ onboarding.py
■   ■■■ profile.py
■   ■■■ analytics.py
■   ■■■ insights.py
■
■■■ tests/
    ■■■ unit/
    ■   ■■■ test_profile_engine.py
    ■   ■■■ test_analytics_engine.py
    ■   ■■■ test_feature_extractor.py
    ■   ■■■ test_intent_classifier.py
    ■■■ integration/
        ■■■ test_api_flows.py

```

3.3 Layered Architecture

The architecture follows a strict layered dependency rule from Clean Architecture:



Why this matters in production:

- You can test your ProfileEngine without spinning up a database.
- You can swap PostgreSQL for a different DB without touching domain logic.
- You can replace the IntentClassifier with an ML model without touching API routes.
- The system becomes independently testable at every layer.

3.4 Monolith vs Microservices Decision

For MVP: monolith. Full stop. Here is the reasoning:

A microservices architecture introduces operational overhead — service discovery, distributed tracing, inter-service authentication, network latency — that is only justified when independent scaling is required, multiple teams need to deploy independently, or services have genuinely different scaling characteristics. At MVP, none of these conditions exist.

A well-structured monolith with clean module boundaries gives you the option to extract services later because your modules already reflect service boundaries. This is called a modular monolith, and it is the professional starting point. When you eventually need to scale, you extract ProfileEngine and AnalyticsEngine into independent services — because your module boundaries already make them independently deployable.

3.5 API Architecture

Your API follows RESTful resource-oriented design with clear semantic versioning:

POST	/api/v1/auth/register	# Create user account
POST	/api/v1/auth/login	# Authenticate, get JWT
POST	/api/v1/onboarding	# Submit onboarding data
GET	/api/v1/profile	# Get my financial profile
GET	/api/v1/profile/history	# Get profile change history
GET	/api/v1/analytics/{user_id}	# Full analytics state

GET	/api/v1/scoring/{user_id}	# All computed scores
GET	/api/v1/dashboard/{user_id}	# Dashboard-ready payload
POST	/api/v1/insights/query	# Ask a financial question
GET	/api/v1/insights/history	# Insight history
GET	/api/v1/vectors/{user_id}	# Binary feature vector
GET	/api/v1/vectors/similar	# Find similar profiles

Rule Engine & Processing Pipeline

4.1 What a Rule Engine Is

A rule engine is a deterministic decision system. Given the same inputs, it always produces the same outputs. There is no randomness, no learning, no hallucination. Just logic. In formal terms it is an implementation of production rule systems from AI research, stripped to its essential form: *if condition then consequence*.

Your system's rule engine serves two purposes:

- Profile Generation — derive structured financial attributes from user inputs.
- Analytics Routing — map profile attributes to analytics categories and scoring weights.

4.2 Rule Abstraction Levels

Amateurs hardcode if-statements. Senior engineers abstract rules into data structures so they are configurable, testable, and extensible. There is a clear progression of architectural thinking here.

Level 1 — Naive (do not do this)

```
def compute_income_level(income):  
    if income < 100000:  
        return 'low'  
    elif income < 300000:  
        return 'medium'  
    else:  
        return 'high'
```

This works but is not extensible. To change thresholds you must change code.

Level 2 — Rule Registry

```
INCOME_LEVEL_THRESHOLDS = [  
    (100_000, 'low'),  
    (300_000, 'medium'),  
    (float('inf'), 'high'),  
]  
  
def classify_income_level(income: int) -> str:  
    for threshold, label in INCOME_LEVEL_THRESHOLDS:  
        if income <= threshold:  
            return label
```

Now thresholds are a configuration concern, not a logic concern.

Level 3 — Rule Objects

```

from dataclasses import dataclass
from typing import Callable

@dataclass
class Rule:
    name: str
    description: str
    condition: Callable[[dict], bool]
    effect: Callable[[dict], dict]
    priority: int = 0

FINANCIAL_PRESSURE_RULES: list[Rule] = [
    Rule(
        name='low_income_rent_pressure',
        description='User with income < 150k paying rent has high financial pressure',
        condition=lambda ctx: ctx['income'] < 150_000 and ctx['pays_rent'] is True,
        effect=lambda p: {**p, 'financial_pressure': 'high'},
        priority=10,
    ),
    Rule(
        name='dependant_pressure',
        description='Any dependants increase financial pressure',
        condition=lambda ctx: ctx['has_dependants'] is True,
        effect=lambda p: {**p, 'pressure_score': p.get('pressure_score', 0) + 2},
        priority=5,
    ),
]

```

Level 4 — Expert: Rule Evaluator with Priority Resolution

```

class RuleEvaluator:
    def __init__(self, rules: list[Rule]):
        self.rules = sorted(rules, key=lambda r: r.priority, reverse=True)

    def evaluate(self, context: dict, initial_profile: dict) -> dict:
        profile = dict(initial_profile)
        for rule in self.rules:
            try:
                if rule.condition(context):
                    profile = rule.effect(profile)
            except Exception as e:
                logger.warning(f"Rule '{rule.name}' failed: {e}")
        return profile

```

Why this architecture matters:

- Rules are prioritised and evaluated in order.
- Rules are independently testable — `rule.condition(some_context)` works in isolation.
- Adding a new rule means adding a new Rule object, not modifying existing logic.
- This is the Open/Closed Principle — open for extension, closed for modification.

4.3 Scoring System Design

Some attributes like `financial_pressure` are not binary — they are scalar. A scoring system accumulates points across multiple rules:

```
PRESSURE_SCORING_RULES = {
    'income_below_100k': 3, # Very high weight
    'pays_rent': 2,
    'has_dependants': 2,
    'income_100k_to_150k': 2,
    'lives_in_lagos': 1,
    'no_savings': 1,
}

PRESSURE_THRESHOLDS = {
    (0, 2): 'low',
    (3, 4): 'medium',
    (5, 6): 'high',
    (7, float('inf')): 'critical',
}

def derive_pressure_label(score: int) -> str:
    for (low, high), label in PRESSURE_THRESHOLDS.items():
        if low <= score <= high:
            return label
    return 'unknown'
```

This scoring model is powerful because:

- Multiple weak signals compound into strong conclusions.
- Individual rules are additive, not exclusive.
- Threshold boundaries are configurable constants.
- You can explain to a user exactly why their pressure score is high — full audit trail.

4.4 The Full Profile Generation Pipeline

```
class ProfileEngine:
    def __init__(self):
        self.evaluator = RuleEvaluator(ALL_PROFILE_RULES)

    def generate_profile(self, user_input: UserInputData) -> FinancialProfile:
        # Step 1: Normalise raw input into context dict
        context = self._normalise_input(user_input)

        # Step 2: Compute base attributes deterministically
        income_level = self._classify_income(context['income'])
        cost_of_living = NigerianContext.get_cost_of_living(context['state'])

        # Step 3: Run rule evaluator for derived attributes
        initial_profile = {
            'income_level': income_level,
            'cost_of_living': cost_of_living,
            'pressure_score': 0,
        }
```

```

evaluated = self.evaluator.evaluate(context, initial_profile)

# Step 4: Post-process scores into labels
evaluated['financial_pressure'] = derive_pressure_label(
    evaluated['pressure_score']
)

# Step 5: Derive investment readiness
evaluated['investment_readiness'] = self._derive_investment_readiness(evaluated)

# Step 6: Derive likely expenses
evaluated['likely_expenses'] = self._derive_expense_profile(context, evaluated)

# Step 7: Set strategic priority
evaluated['priority'] = self._derive_priority(evaluated)

return FinancialProfile(**evaluated)

```

4.5 Nigerian Context as a First-Class Domain Object

The Nigerian financial knowledge base should not live in scattered if-statements. It deserves its own domain module. This makes your domain knowledge addressable, testable, and updatable independently of logic.

```

# domain/knowledge/nigerian_context.py

COST_OF_LIVING_BY_STATE = {
    'Lagos': 'very_high',
    'Abuja': 'high',
    'Ibadan': 'medium',
    'Kano': 'medium',
    'Enugu': 'medium',
    'Imo': 'low',
    'Ekiti': 'low',
}

AVERAGE_MONTHLY_TRANSPORT_BY_STATE = {
    'Lagos': 15_000, # BRT + ride-hailing + okada
    'Abuja': 10_000,
    'Ibadan': 6_000,
}

TYPICAL_RENT_RATIO_BY_INCOME_LEVEL = {
    'low': 0.30, # 30% of annual income as monthly equivalent
    'medium': 0.20,
    'high': 0.10,
}

class NigerianContext:
    @staticmethod
    def get_cost_of_living(state: str) -> str:
        return COST_OF_LIVING_BY_STATE.get(state, 'medium')

```

```
@staticmethod
def estimated_base_expenses(income: int, state: str, pays_rent: bool) -> dict:
    transport = AVERAGE_MONTHLY_TRANSPORT_BY_STATE.get(state, 7_000)
    food = income * 0.25
    data_sub = 3_000
    rent_monthly = income * 0.20 if pays_rent else 0
    return {
        'transport': transport,
        'food': int(food),
        'data': data_sub,
        'rent_equiv': int(rent_monthly),
    }
```

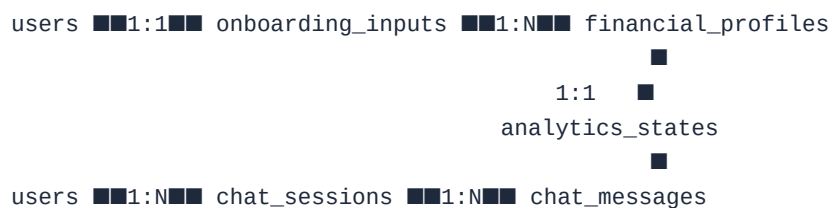
Database Design

5.1 Relational Modelling Principles

Your data model must support:

- User identity and authentication
- Onboarding input storage for audit and reprocessing
- Generated financial profiles with versioning
- Analytics states and computed scores
- Chat sessions and message history
- Advisory insight history

5.2 Entity-Relationship Design



5.3 SQLAlchemy Schema

```

class FinancialProfile(Base):
    __tablename__ = 'financial_profiles'

    id = Column(UUID, primary_key=True, default=uuid.uuid4)
    user_id = Column(UUID, ForeignKey('users.id'), nullable=False)
    input_id = Column(UUID, ForeignKey('onboarding_inputs.id'))
    income_level = Column(String(20)) # low | medium | high
    risk_level = Column(String(20))
    financial_pressure = Column(String(20)) # low | medium | high | critical
    saving_capacity = Column(String(20))
    investment_readiness = Column(String(30))
    priority = Column(String(50))
    likely_expenses = Column(JSON) # ['rent', 'food', 'transport']
    profile_vector = Column(JSON) # Binary feature vector
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)

class AnalyticsState(Base):
    __tablename__ = 'analytics_states'

    id = Column(UUID, primary_key=True, default=uuid.uuid4)
  
```

```
profile_id          = Column(UUID, ForeignKey('financial_profiles.id'))
user_id             = Column(UUID, ForeignKey('users.id'))
pressure_score      = Column(Integer)
savings_rate        = Column(Float)
debt_to_income_ratio = Column(Float)
emergency_fund_gap   = Column(Integer) # In naira
investment_score     = Column(Float)
overall_health_score = Column(Float)
computed_at         = Column(DateTime, default=datetime.utcnow)
```

5.4 Why Profile Versioning Matters

Financial profiles store a new row every time the profile is recomputed. The `is_active` flag identifies the current profile. Historical profiles are never deleted.

- You can trace how a user's financial situation changed over time.
- You can audit why specific advice was given — based on which profile was active at that moment.
- When you introduce ML, this historical data becomes your training set.
- It prevents data loss from re-onboarding.

Object-Oriented Design & SOLID Principles

6.1 Polymorphism in the System

Polymorphism means the same interface, multiple implementations. You call the same method but get different behaviour depending on which concrete class is behind it. In this system, polymorphism shows up in at least three places.

6.1.1 The Insight Renderer (Strategy Pattern)

Different insight questions require different rendering strategies. Today you use templates. Tomorrow you use an LLM. The code that calls the insight engine must not need to change.

```
# domain/insights/base.py
from abc import ABC, abstractmethod

class InsightRenderer(ABC):
    @abstractmethod
    def render(self, analytics: AnalyticsState, intent: str) -> FinancialInsight:
        raise NotImplementedError

# domain/insights/template_renderer.py
class TemplateInsightRenderer(InsightRenderer):
    def render(self, analytics: AnalyticsState, intent: str) -> FinancialInsight:
        template = INSIGHT_TEMPLATES[intent]
        return template.fill(analytics)

# domain/insights/llm_renderer.py (FUTURE)
class LLMInsightRenderer(InsightRenderer):
    def __init__(self, api_client: ClaudeClient):
        self.client = api_client

    def render(self, analytics: AnalyticsState, intent: str) -> FinancialInsight:
        # Backend-computed truth → LLM explains it
        truth = analytics.to_structured_dict()
        prompt = self._build_explanation_prompt(truth, intent)
        return self.client.complete(prompt)
```

When you want to upgrade from templates to LLM, you swap one line in your dependency injection setup. Zero changes to InsightEngine. Zero changes to API routes. That is the power of programming to an interface, not an implementation.

6.1.2 The Profile Evaluator (Strategy Pattern for Rules)

```
class ProfileEvaluationStrategy(ABC):
    @abstractmethod
    def evaluate(self, context: dict) -> FinancialProfile:
```

```

...

class SalariedWorkerStrategy(ProfileEvaluationStrategy):
    def evaluate(self, context: dict) -> FinancialProfile:
        # Rules optimised for salaried workers
        ...

class GigWorkerStrategy(ProfileEvaluationStrategy):
    def evaluate(self, context: dict) -> FinancialProfile:
        # Rules optimised for irregular income
        ...

```

6.1.3 The Intent Classifier (Strategy Pattern for NLP)

```

class IntentClassifier(ABC):
    @abstractmethod
    def classify(self, text: str) -> str:
        ...

class KeywordIntentClassifier(IntentClassifier):
    INTENT_KEYWORDS = {
        'loan_query':      ['loan', 'borrow', 'credit'],
        'investment_query': ['invest', 'stock', 'crypto'],
        'savings_query':   ['save', 'saving', 'emergency fund'],
        'budget_query':    ['budget', 'spend', 'allocate'],
    }
    def classify(self, text: str) -> str:
        text_lower = text.lower()
        for intent, keywords in self.INTENT_KEYWORDS.items():
            if any(kw in text_lower for kw in keywords):
                return intent
        return 'general_advice'

class MLIntentClassifier(IntentClassifier):
    def classify(self, text: str) -> str:
        return self.model.predict(text)

```

6.2 SOLID Principles Applied

Principle	Application in This Project
Single Responsibility	ProfileEngine only generates profiles. AnalyticsEngine only computes metrics.
Open/Closed	Adding a new rule = add a new Rule object. Never modify the evaluator.
Liskov Substitution	Any InsightRenderer subclass can replace another without breaking the engine.
Interface Segregation	InsightRenderer only exposes render(). No unnecessary methods exposed.
Dependency Inversion	InsightEngine depends on the InsightRenderer abstraction, not a concrete implementation.

6.3 Dependency Injection in FastAPI

```
# main.py
def get_insight_renderer() -> InsightRenderer:
    return TemplateInsightRenderer() # Swap to LLMInsightRenderer in future

def get_insight_engine(
    renderer: InsightRenderer = Depends(get_insight_renderer)
) -> InsightEngine:
    return InsightEngine(renderer=renderer)

@router.post('/insights/query')
def query_insight(
    question: InsightRequest,
    engine: InsightEngine = Depends(get_insight_engine),
    db: Session = Depends(get_db)
):
    analytics = db.query(AnalyticsState).filter_by(
        user_id=current_user.id, is_active=True
    ).first()
    insight = engine.answer(analytics, question.text)
    return insight
```

Analytics & Scoring APIs

7.1 API Architecture Overview

The backend exposes six distinct API categories. Each serves a different consumer of the analytics data — the dashboard, the insight chat interface, and future ML systems.

7.2 Financial Input APIs

These APIs collect and validate raw financial data. They trigger the processing pipeline.

POST /api/v1/onboarding

Request:

```
{
  'age_range': '25-34',
  'state': 'Lagos',
  'occupation': 'private_sector',
  'monthly_income_range': '75000-100000',
  'has_dependants': true,
  'pays_rent': true,
  'bank': 'GTBank'
}
```

Response: 201 Created

```
{
  'profile_id': 'uuid',
  'analytics_state_id': 'uuid',
  'income_level': 'low',
  'financial_pressure': 'high',
  'investment_readiness': 'not_ready',
  'priority': 'stability'
}
```

7.3 Analytics & Scoring APIs

These APIs serve computed scores and metrics. They never recompute — they read from the stored AnalyticsState.

GET /api/v1/scoring/{user_id}

Response:

```
{
  'pressure_score': 7,
  'pressure_label': 'critical',
  'savings_rate': 0.04,
  'debt_to_income_ratio': 0.65,
  'emergency_fund_gap': 240000,
  'investment_score': 12.5,
  'overall_health_score': 31.2,
}
```

```
    'computed_at': '2025-01-15T09:00:00Z'
  }
```

7.4 Dashboard APIs

Dashboard APIs serve pre-formatted analytics payloads optimised for direct consumption by chart and summary components. The frontend should never compute — it only renders.

```
GET /api/v1/dashboard/{user_id}
Response:
{
  'summary': {
    'headline': 'Your finances are under critical pressure',
    'priority_action': 'Build emergency buffer before any investment'
  },
  'expense_breakdown': [
    {'category': 'Rent', 'amount': 17500, 'percentage': 20},
    {'category': 'Food', 'amount': 21875, 'percentage': 25},
    {'category': 'Transport', 'amount': 15000, 'percentage': 17}
  ],
  'scores': { ... },
  'timeline': [
    {'month': '2024-10', 'pressure_score': 8},
    {'month': '2024-11', 'pressure_score': 7}
  ]
}
```

7.5 Insight APIs

Insight APIs handle natural language queries against the user's analytics state. The backend classifies the intent, selects the appropriate insight template, personalises it with real metrics, and returns a structured response.

```
POST /api/v1/insights/query
Request:
{ 'question': 'Can I start investing this month?' }

Response:
{
  'intent': 'investment_readiness_query',
  'readiness': 'not_ready',
  'reasoning': [
    'Your emergency fund gap is ■240,000',
    'Your pressure score is 7 (critical threshold)',
    'You have 0 months of buffer savings'
  ],
  'recommendation': 'Focus on 3-month buffer before considering investment.',
  'next_milestone': 'Save ■80,000 over the next 3 months'
}
```

Binary Features & Vectorization Architecture

8.1 Why Binary Representation Matters

A binary feature represents the presence or absence of a condition: 1 or 0. When you combine many binary features into a vector, you get a mathematical representation of an entity that can be compared to other entities (user similarity), fed into ML models directly, used to compute scores via weighted dot products, stored compactly, and queried efficiently.

This is the bridge between your deterministic rule engine and your future ML systems. You are not adding ML on top of the system. You are building the system so that ML is a natural next step.

8.2 The Profile Feature Vector

```

PROFILE_FEATURES = [
    'income_is_low',          # income < 150k
    'income_is_medium',      # 150k <= income < 400k
    'income_is_high',        # income >= 400k
    'pays_rent',              # Boolean
    'has_dependants',         # Boolean
    'lives_in_high_cost_city', # Lagos, Abuja
    'is_young_adult',         # 18-34
    'occupation_formal',      # salaried / formal sector
    'occupation_informal',    # gig / self-employed
    'high_financial_pressure', # pressure_score >= 5
    'has_savings_capacity',    # can set aside any amount
    'investment_ready',        # investment_readiness != 'not_ready'
]

def compute_feature_vector(context: dict, profile: dict) -> dict:
    income = context['income']
    return {
        'income_is_low':      int(income < 150_000),
        'income_is_medium':   int(150_000 <= income < 400_000),
        'income_is_high':     int(income >= 400_000),
        'pays_rent':          int(context['pays_rent']),
        'has_dependants':      int(context['has_dependants']),
        'lives_in_high_cost_city': int(context['state'] in ['Lagos', 'Abuja']),
        'is_young_adult':      int(context['age_range'] in ['18-24', '25-34']),
        'occupation_formal':    int(context['occupation'] == 'private_sector'),
        'occupation_informal':  int(context['occupation'] in ['self_employed', 'trader']),
        'high_financial_pressure': int(profile.get('pressure_score', 0) >= 5),
        'has_savings_capacity': int(profile.get('saving_capacity') not in ['none']),
        'investment_ready':     int(profile.get('investment_readiness') != 'not_ready'),
    }

```

8.3 Weighted Scoring for Insight Priority

```

INSIGHT_WEIGHTS = {
    'emergency_fund_advice': [1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0],
    'investment_advice':     [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1],
    'debt_management_advice': [1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0],
}

def score_insight_relevance(user_vector: list, insight_weights: list) -> float:
    return sum(u * w for u, w in zip(user_vector, insight_weights))

```

This dot product gives you a relevance score for each possible insight. Higher score = more relevant for this user's profile.

8.4 Generic Adaptability

The feature vector schema is designed so that extending the system requires no architectural surgery:

- Adding a new feature = add one key to PROFILE_FEATURES and one computation to compute_feature_vector.
- Adding a new insight category = add one weight vector to INSIGHT_WEIGHTS.
- Adding a new Nigerian state = add one entry to COST_OF_LIVING_BY_STATE.
- None of these changes require modifying existing logic. They are purely additive.

Analytical queries become trivial once vectors are stored in PostgreSQL JSONB:

```

-- Find all users who pay rent AND have dependants AND are low income
SELECT user_id
FROM financial_profiles
WHERE (profile_vector->>'pays_rent')::int = 1
    AND (profile_vector->>'has_dependants')::int = 1
    AND (profile_vector->>'income_is_low')::int = 1
    AND is_active = true;

```

Future-Proofing & ML Readiness

9.1 LLM Integration Architecture

The critical architectural rule is already stated: the LLM never computes financial truth. The backend computes truth. The LLM only explains truth.

This means the LLM integration layer sits entirely outside the analytics computation pipeline. It receives a fully computed `AnalyticsState` and translates it into natural language. The LLM is a rendering concern, not a financial reasoning concern.

```
# The backend computes:
analytics = AnalyticsEngine.compute(profile)
# {
#   'pressure_score': 7,
#   'emergency_fund_gap': 240000,
#   'investment_readiness': 'not_ready'
# }

# The LLM only explains:
prompt = f"""
You are a Nigerian financial advisor assistant.
The backend has computed this user's financial state:
{json.dumps(analytics.to_dict())}

Explain this situation clearly to the user.
Do not invent or change any numbers.
Only explain what is given.
"""
```

9.2 pgvector & Vector Databases

When your user base grows and you want recommendation systems and user similarity search, you add pgvector to PostgreSQL. No new database required.

```
-- Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Add embedding column to financial_profiles
ALTER TABLE financial_profiles
ADD COLUMN profile_embedding vector(12);

-- Find the 5 users most similar to a given user
SELECT user_id, 1 - (profile_embedding <=> '[1,0,1,1,0,1,1,0,0,1,0,0]') AS similarity
FROM financial_profiles
WHERE is_active = true
ORDER BY profile_embedding <=> '[1,0,1,1,0,1,1,0,0,1,0,0]'
LIMIT 5;
```

Your binary feature vectors are immediately usable as pgvector embeddings. You built vector-readiness into the system from day one.

9.3 Event-Driven Systems

As the system scales, certain operations should become asynchronous. Profile generation and analytics computation are compute operations that should not block API responses.

```
# Future: Celery task for async profile computation
@celery.task
def compute_profile_async(user_id: str, input_id: str):
    user_input = onboarding_repo.get(input_id)
    profile = ProfileEngine().generate_profile(user_input)
    profile_repo.save(profile)
    analytics = AnalyticsEngine().compute(profile)
    analytics_repo.save(analytics)
    notify_user_profile_ready(user_id)

# API route becomes non-blocking:
@router.post('/onboarding')
def submit_onboarding(data: OnboardingInput):
    input_record = onboarding_repo.save(data)
    compute_profile_async.delay(current_user.id, input_record.id)
    return {'status': 'processing', 'input_id': input_record.id}
```

9.4 Observability Strategy

Production systems must be observable. Instrument these events from day one:

```
logger.info('profile_generated', extra={
    'user_id': str(user_id),
    'income_level': profile.income_level,
    'financial_pressure': profile.financial_pressure,
    'rules_triggered': profile.rules_triggered,
    'duration_ms': elapsed_ms,
})

logger.info('analytics_computed', extra={
    'user_id': str(user_id),
    'pressure_score': analytics.pressure_score,
    'health_score': analytics.overall_health_score,
    'profile_snapshot': profile.id,
})

logger.info('insight_rendered', extra={
    'user_id': str(user_id),
    'intent': intent,
    'template_id': template_id,
    'renderer': renderer.__class__.__name__,
})
```

9.5 Testing Strategy

Test Type	What to Test
Unit — Rules	Each rule's condition function with various contexts
Unit — Engine	ProfileEngine with known inputs → expected profile attributes
Unit — Analytics	AnalyticsEngine with mock profiles → expected metric values
Unit — Intent	IntentClassifier with known strings → expected intent labels
Unit — Vector	compute_feature_vector with known contexts → expected binary outputs
Integration	Full API flow: POST /onboarding → GET /analytics → POST /insights/qu
Regression	Every bug fix adds a test that would have caught it

9.6 Migration Path to ML

This system is structured so that adding machine learning is an additive step, not a destructive rewrite:

- Data is being collected — every profile generation and Q&A; is stored with timestamps and audit data.
- Profile vectors exist — immediately usable as ML features without any transformation.
- Intent classifier has a swappable interface — replace keyword classifier with a fine-tuned model.
- InsightRenderer has a swappable interface — replace template renderer with LLM renderer.
- Profile Engine rules become training labels — supervised learning against labeled outcomes.

The MVP is simultaneously a production system and a data flywheel for future ML. You are not building tech debt. You are building runway.

Implementation Roadmap

Phase 1 — Foundation (Week 1–2)

Engineering priority: get the data layer and auth right. Everything builds on this.

- FastAPI project scaffolded with the proper directory structure
- SQLAlchemy models created and migrations working with Alembic
- User authentication working — JWT, bcrypt
- POST /auth/register and POST /auth/login endpoints active

Phase 2 — Profile Engine (Week 3–4)

Engineering priority: this is the intellectual core. Test every rule independently.

- Onboarding input schema validated via Pydantic
- All profile rules implemented as Rule objects
- ProfileEngine generates FinancialProfile deterministically
- Binary feature vector computed and stored in JSONB
- POST /onboarding and GET /profile endpoints working
- Unit tests for every rule

Phase 3 — Analytics Engine (Week 5)

Engineering priority: make every metric computable and auditable.

- AnalyticsEngine computing all financial metrics from profile
- AnalyticsState persisted to DB on every profile generation
- GET /analytics and GET /scoring endpoints working

Phase 4 — Insight Engine (Week 6)

Engineering priority: get the Q&A; loop working. Templates can be refined later.

- Intent classifier implemented — keyword-based MVP
- Insight template registry — 5–10 templates minimum
- InsightEngine connecting classifier → analytics → template → response
- POST /insights/query endpoint working end-to-end

Phase 5 — Dashboard API & Chat History (Week 7)

- Dashboard API returning pre-formatted analytics payloads
- Chat sessions tracked and stored
- Message history retrievable under sessions
- Sessions linked to profile snapshot for auditability

Phase 6 — Frontend Integration (Week 8–9)

- React app with routing
- Onboarding form — under 60 seconds UX target
- Analytics dashboard rendering from API payloads
- Insight chat interface for Q&A;
- API integration complete

Phase 7 — Polish and Production (Week 10)

- Full integration test suite
- Nigerian financial knowledge base reviewed for accuracy
- Insight templates reviewed for cultural accuracy
- Error handling hardened
- Deployment — Railway, Render, or VPS

Quick Reference

Complete Rule Set — MVP Starter

```
INCOME_THRESHOLDS = {
    'very_low': 50_000,
    'low':      150_000,
    'medium':   400_000,
    'high':     float('inf'),
}

CORE_RULES = [
    # INCOME PRESSURE RULES
    Rule('critical_poverty_line',
        condition=lambda c: c['income'] < 50_000,
        effect=lambda p: {'p', 'priority': 'survival', 'pressure_score': p.get('pressure_score',0)+3}),

    Rule('low_income_rent',
        condition=lambda c: c['income'] < 150_000 and c['pays_rent'],
        effect=lambda p: {'p', 'pressure_score': p.get('pressure_score',0)+3}),

    Rule('dependant_load',
        condition=lambda c: c['has_dependants'],
        effect=lambda p: {'p', 'pressure_score': p.get('pressure_score',0)+2}),

    Rule('lagos_cost_multiplier',
        condition=lambda c: c['state'] == 'Lagos',
        effect=lambda p: {'p', 'pressure_score': p.get('pressure_score',0)+1}),

    # INVESTMENT READINESS RULES
    Rule('investment_eligible',
        condition=lambda c: c['income'] >= 400_000 and not c['has_dependants'],
        effect=lambda p: {'p', 'investment_readiness': 'ready'}),

    Rule('investment_developing',
        condition=lambda c: 200_000 <= c['income'] < 400_000,
        effect=lambda p: {'p', 'investment_readiness': 'developing'}),

    Rule('investment_blocked_dependants',
        condition=lambda c: c['has_dependants'] and c['income'] < 200_000,
        effect=lambda p: {'p', 'investment_readiness': 'not_ready'}),

    # SAVINGS CAPACITY RULES
    Rule('no_savings_capacity',
        condition=lambda c: c['income'] < 80_000 and c['pays_rent'],
        effect=lambda p: {'p', 'saving_capacity': 'none'}),

    Rule('micro_savings',
        condition=lambda c: 80_000 <= c['income'] < 150_000,
```

```

        effect=lambda p: {**p, 'saving_capacity': 'micro'})), # 2,000-5,000/month
    ]

```

Insight Template Example

```

INSIGHT_TEMPLATES = {
    'budget_allocation': {
        'low_income_high_pressure': """
Based on your financial profile, here is a practical allocation:

SURVIVAL FIRST (70-80% = N{survival_amount:,})
    Rent equivalent:  N{rent_est:,}/month
    Food:             N{food_est:,}
    Transport:        N{transport_est:,}
    Data:             N3,000

EMERGENCY BUFFER (10-15% = N{buffer_amount:,})
    Target: 3 months of survival expenses = N{emergency_target:,}
    At this rate, you reach it in {months_to_target} months.

FLEXIBLE (5-10% = N{flex_amount:,})

REALITY CHECK:
You are in a financial pressure situation. Investment is not
your next step. Stability is. Once your emergency fund is
built, we revisit your priorities.
"""
    }
}

```

How to Sound Like an Architect

Junior engineers describe what they built. Senior engineers explain why they made the decisions they made and what trade-offs they accepted. For every decision in your system, be ready to answer three questions:

- **Why did you choose this?** Not just 'it was easy.' What specific properties of this choice served the requirements?
- **What did you sacrifice?** Every architectural decision is a trade-off. Monolith sacrifices independent scalability; microservices sacrifice operational simplicity. Name the trade-off.
- **How does this change as the system scales?** What breaks first? What would you change at 10,000 users? At 1 million?

Example of architect-level framing for the core design choice in this system:

Junior framing:

'I used a rule engine because it was simpler than ML.'

Architect framing:

'I chose a deterministic rule engine over an ML-based approach because financial advisory systems require explainability by design. A user

must be able to ask why their pressure score is 7, and the system must be able to answer that question with an audit trail of which rules fired and with what weights. ML gives you accuracy but not explainability at this granularity. The rule engine gives you both correctness and explainability, with a clear migration path to ML once sufficient labelled data exists. The binary feature vector we store today is the training data for that future model.'

This document is a complete backend architecture blueprint for a production-grade Nigerian Financial Intelligence and Analytics Engine. Every decision is justified by engineering principles, domain constraints, and scalability thinking appropriate to a senior systems architect.